



SAVEETHA
INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Declared as Deemed to be University under Section 3 of UGC Act 1956)



SIMATS ENGINEERING

Descriptive Written Exam

COURSE NAME: Software Testing

COURSE CODE: CSA37

COURSE OUTCOME COVERED

CO1: *Apply software testing principles to enhance the effectiveness of software quality assurance and improve defect detection strategies.*

Assessment Tool Weightage: 50%

Code of Conduct:

*I **Lingesh B (192421149)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand

Q1. Testing as an Engineering Activity

In a bank, testing has to be run like an engineering discipline: planned, measured and continuously improved. Errors here touch money, personal data and customer trust, so a few checks after coding are not enough.

Requirements → Risk identification → Test planning → Test design → Execution → Defect reporting → Retest → Regression

HOW THE TEAM SHOULD WORK

- Start from the requirements and separate functional from non-functional needs; rank login, balance, fund transfer, withdrawal and history as top priority.
- Turn every requirement into test conditions, then into test cases whose **expected results are fixed before running** them.
- Keep cases in a test-management tool and log every defect in one shared tracker.
- Automate the repetitive regression scenarios.

TECHNIQUES THAT REPLACE AD-HOC CHECKING

Technique	What it contributes
Equivalence Partitioning	Groups inputs into valid and invalid classes
Boundary Value Analysis	Probes the limits and the values around them
Decision Table Testing	Covers combinations of conditions and actions
State Transition Testing	Checks movement between transaction states
Risk-Based Testing	Puts effort on the riskiest banking operations
Regression Testing	Confirms changes have not broken existing behaviour

Worked example – fund transfer: Do not stop at the successful transfer. Also try insufficient balance, an invalid beneficiary, the maximum limit, a duplicate request, a network drop, a timeout and an unauthorised attempt. With ₹25,000 in the account, a ₹10,000 transfer must leave exactly ₹15,000 and add one correct entry to the history.

Result: better defect detection, requirement coverage, repeatability, traceability, regression strength and overall reliability.

Takeaway: *Treating testing as engineering turns random checking into planned quality assurance, and that measurably cuts defects in banking software.*

Q2. Structured Testing Process Across the SDLC

An ATM handles authentication, account data and cash, so testing must run through every SDLC phase and not wait until coding finishes.

PHASE BY PHASE

SDLC phase	Testing activity
Requirements & planning	Review card validation, PIN check, account verification and balance enquiry; catch vague or missing requirements early
Test design	Write positive and negative cases from the finalised requirements
Execution	Compare actual with expected results; log every mismatch as a defect
After fixes	Retest the fix, then run regression on neighbouring functions

SAMPLE ATM WITHDRAWAL CASES

Condition	Expected outcome
Valid card and PIN	Transaction proceeds
Wrong PIN	Transaction rejected
Balance too low	Withdrawal refused
Amount over limit	Transaction rejected
Invalid amount	Clear error message shown
ATM short of cash	Handled safely
Network failure	No wrong deduction from the account

Why it pays off: An unclear withdrawal limit spotted during requirement review costs almost nothing; the same gap found after deployment is expensive. A consistent process also gives repeatable tests, fewer gaps, clear responsibility and more confidence at release.

Takeaway: *Testing built into each SDLC stage gives steady quality control and improves accuracy, security and reliability of ATM software.*

Q3. Errors, Defects and Failures

The three words describe three different points in the life of a problem, and mixing them up weakens both testing and defect reports.

Human error → Defect in the code → Execution → Failure

Term	Meaning
Error	A mistake by a person during requirements, design or coding
Defect	The flaw left in the software because of that mistake
Failure	The wrong behaviour seen when the flawed code runs

Example 1 – password rule: The requirement says at least eight characters, but the developer misreads it and codes five. The misreading is the **error**, the wrong length check is the **defect**, and the system accepting a five-character password is the **failure**.

Example 2 – transfer: The user sends ₹5,000 and ₹6,000 is deducted. The tester sees a failure that points to a defect in transaction processing.

WHAT A GOOD DEFECT REPORT CONTAINS

- Title, environment and preconditions
- Test data and steps to reproduce
- Expected versus actual result
- Severity, priority, plus logs or screenshots

A note such as “transaction is wrong” gives developers nothing to act on; precise terminology speeds up root-cause analysis.

Takeaway: *Errors come from people, defects live in the software, and failures are what users observe; keeping them apart makes reporting clearer.*

Q4. Defect Clustering and Test Prioritisation

Defect clustering says that a small share of modules usually holds a large share of the defects, so testing effort should follow that concentration.

SPOTTING DEFECT-PRONE MODULES

- High defect counts or severe defects in past releases
- Complicated logic and frequent changes
- Many dependencies and heavy customer usage

In online banking, fund transfer will usually attract more defects than a profile page because of its validation rules and integrations.

PRIORITY SCHEME

Level	Scope
Priority 1 – Critical	Payments, fund transfer, authentication
Priority 2 – Important	Account statements, notifications
Priority 3 – Low risk	Cosmetic or rarely used features

Worked example: If history shows 60% of serious defects came from the payment module, extend coverage there: successful and failed payments, duplicate payments, timeouts, network interruption, invalid card details and refunds.

Benefits: limited effort is focused, serious defects surface sooner and low-risk areas consume less time. Quieter modules still need basic testing, because few past defects does not mean none remain.

Takeaway: *Combine defect history with risk, complexity and business value to decide where testing should be heaviest.*

Q5. Defect Classification and Repository Utilisation

If the same defects return release after release, the organisation is storing its defect history without learning from it.

CATEGORIES OF DEFECTS

Origin	Type
Development stage	Requirement, design, coding
System interaction	Integration, interface, data
Quality attribute	Performance, security

WHAT EACH REPOSITORY RECORD NEEDS

Defect ID Module Description Severity Priority Root cause Status Resolution

Reading these records across releases exposes patterns; for instance, repeated payment-calculation defects show payment processing is fragile.

STOPPING RECURRENCE

- Do root-cause analysis on important defects
- Convert them into regression test cases
- Tighten requirement, design and code reviews

- Test defect-prone modules more and automate repeated checks

Takeaway: *Classification and analysis make the repository a knowledge base that prevents the same defects from reappearing.*

Q6. Mobile Banking Application – Fund Transfer Crashes

Repeated crashes in a critical function suggest testing has not covered enough mixes of transaction, device, network and app conditions.

STEP 1 – GATHER FACTS

Record device model, OS and app version, network state, balance, amount, beneficiary, the exact steps before the crash, error messages and app and server logs. Reproduce it several times to see whether it is consistent.

STEP 2 – TEST THE RIGHT AREAS

Test type	Focus
Functional	Normal and abnormal transfers
Integration	App, authentication service, transaction service and database working together
Network	Weak signal, drops, reconnection, timeouts
Performance	Many simultaneous transactions
Regression	Fixes leave existing functions intact

Worked example: The app crashes when the user taps Transfer just as the network changes from strong to weak. Reproduce that exact switch and read the logs. Likely causes: poor exception handling, a timeout-handling bug, a wrong transaction state or an API communication fault.

IMPROVEMENTS TO INTRODUCE

- Automated regression and network-condition tests
- Crash monitoring and device-compatibility testing
- Exception-handling and transaction-recovery tests

Takeaway: *A methodical mix of functional, integration, network, performance and regression testing exposes the real trigger and stabilises transfers.*

Q7. Testing and Debugging

The two activities are linked but separate: testing shows that a failure exists, debugging finds and removes its cause.

Aspect	Testing	Debugging
Goal	Detect failures	Locate causes
Usually done by	Testers	Developers
Tools	Test cases	Code analysis, logs, debuggers
Output	Defect reports	Corrections
Looks at	Expected behaviour	Internal behaviour

Worked example: An ATM deducts ₹5,000 but pays out only ₹4,000. The tester spots and reports the mismatch. The developer then examines the dispensing logic, the amount calculation or the hardware link, and fixes it. After that: retest, then regression test.

RISKS OF CONFUSING THEM

- Unclear ownership of defects
- Incomplete investigation and poor communication
- Repeated defects and thin regression testing

Test → Detect failure → Report → Debug → Fix → Retest → Regression test

Takeaway: *Keep both roles distinct but coordinated: testing proves the problem, debugging removes it.*

Q8. Cross-Browser Testing for an E-Commerce Platform

Shoppers use many browsers, versions, operating systems and screens, and differences in rendering engines, JavaScript and CSS can make the site behave inconsistently.

STRATEGY

Begin with a compatibility matrix listing every supported browser and version, then run the key journeys on each: login, search, product display, cart, checkout, payment, order confirmation and profile.

TYPICAL SOURCES OF DEFECTS

- JavaScript incompatibility and CSS rendering differences
- Browser-specific features and responsive-layout problems
- Third-party components and version differences

Worked example: The checkout button works in one browser but is dead in another. Reproduce it, note browser and version, check console errors, inspect network requests, compare with a supported browser, and save screenshots as evidence.

WIDENING COVERAGE

- Automated browser tests and cloud-based test environments
- Responsive and exploratory manual testing
- Regression runs after each fix

Takeaway: *Cross-browser testing gives every user consistent behaviour; automation plus manual checks and a matrix provides solid coverage.*

Q9. Defect Classification in Payroll Systems

Wrong salary figures from faulty program logic are mainly a coding (logic) defect, but testers must also check whether the root lies in the requirement or design.

Level	Possible source
Requirement	Salary rules specified incorrectly
Design	Wrong calculation model
Coding	Formula implemented with a logic error
Data	Incorrect salary or deduction data processed

SUITABLE TECHNIQUES

- **Equivalence Partitioning** – group employees by salary band or employment status
- **Boundary Value Analysis** – salary, tax and deduction limits
- **Decision Tables** – pay that depends on several conditions
- **Regression Testing** – fixes must not disturb other calculations

Worked example: Basic ₹30,000 + allowances ₹5,000 – deductions ₹3,000 should give a net salary of ₹32,000. If the system shows ₹28,000, trace the calculation logic to find the defect.

Prevention: requirement and code reviews, unit tests, boundary tests and automated calculation checks.

Takeaway: *Classifying the defect correctly guides the choice of technique and keeps payroll output accurate.*

Q10. Structured Test Case Design

Random testing may expose obvious failures, but it cannot show that important requirements were checked. Cases should be designed before they are run.

ELEMENTS OF A GOOD TEST CASE

ID	Objective	Preconditions	Input data	Steps	Expected result	Actual result	Pass/Fail	Defect reference
----	-----------	---------------	------------	-------	-----------------	---------------	-----------	------------------

DESIGN TECHNIQUES

Equivalence Partitioning, Boundary Value Analysis, Decision Tables, State Transition, Error Guessing and Risk-Based Testing.

EXAMPLE – LOGIN FUNCTION

Condition	Expected behaviour
Valid username and password	Login succeeds
Invalid username	Rejected
Invalid password	Rejected
Empty fields	Validation message
Locked account	Access denied
Repeated wrong attempts	Suitable security response

Gains: wider requirement coverage, repeatable runs, clear expected results, easier defect spotting, stronger regression and better traceability.

Takeaway: *Structured design makes testing a planned engineering activity and gives measurable evidence of quality.*

Q11. Data Integrity in Healthcare Applications

Registration, appointments, laboratory, pharmacy, billing and medical records are tightly connected, so inconsistent data across modules weakens reliability.

WHERE SUCH DEFECTS COME FROM

- Wrong requirements or database design
- Incorrect field mapping and duplicate records
- Integration errors, faulty update logic and data transformation problems

VALIDATION APPROACH

Registration → Appointment → Laboratory → Pharmacy → Billing

Follow one patient through the whole chain and confirm that patient ID, name, date of birth, test details and billing data stay correct at each step.

Worked example: The date of birth is right at registration but different in billing. Establish whether the fault arose in data storage, transfer, retrieval, conversion or only in the user-interface display.

IMPROVEMENTS

- Integration testing and database validation
- Data-consistency checks and regression testing
- Detailed defect reports and requirement traceability

Takeaway: *Following data end to end keeps healthcare records accurate between modules.*

Q12. Defect Repository and Recurring Defects

Recurring defects mean past testing information is not being used. A repository should feed new preventive tests.

RECORD FOR EVERY DEFECT

Defect ID | Module | Description | Severity | Priority | Root cause | Status | Resolution

PATTERNS WORTH EXAMINING

- Defects by module, severity, release and root cause
- Defect types that keep returning

Worked example: If discount-related defects keep appearing in an e-commerce system, add tests for many discount combinations.

PREVENTIVE ACTIONS

- Add important past defects to the regression suite
- Run root-cause analysis and improve requirement reviews
- Test defect-prone modules harder and review old defects before each release
- Automate key regression scenarios

Takeaway: *Use the repository as a source of testing knowledge, not just storage, to stop repeat defects.*

Q13. Peak-Usage Failures in a Web-Based Learning System

A platform that works for a few users may collapse during exams, result publication or assignment deadlines. This points to weak performance and workload testing.

PROBABLE GAPS

- Too little load, stress or concurrent-user testing
- Unrealistic test data and shallow database testing
- Poor monitoring and no recovery scenarios

Test type	Purpose
Load	Behaviour at expected user volume

Test type	Purpose
Stress	Behaviour beyond normal capacity
Performance	Response time, throughput, resource use
Recovery	Ability to recover after overload or failure

Worked example: An exam system copes with 100 users but fails when 5,000 students submit together. Watch response time, CPU, memory, database performance, network activity, error rate and transaction completion.

Action: build realistic peak-load scenarios, simulate large user numbers with automated performance tests, and repeat them before every major release.

Takeaway: Failures at peak time arise when real workloads are missing from testing; load, stress, performance and recovery tests close that gap.

Q14. Early Involvement of Testers in the SDLC

Bringing testers in only after development wastes the chance to remove ambiguity and defects while they are cheap to fix.

Stage	What testers do
Requirements	Review requirements, spot ambiguity, check testability, find missing conditions
Design	Review workflows, integration risks and data flow; anticipate failure conditions
Development	Prepare test data, support unit-test planning, identify automation chances
Testing	Execute tests, report defects, retest and run regression

Worked example – transfer requirement: An early tester asks: What if the balance is too low? The beneficiary account is invalid? The network fails? The user submits twice? The limit is exceeded? Answering these before coding clarifies the requirement.

Benefits: earlier detection, lower rework cost, better requirements and testability, sharper risk identification and closer teamwork.

Takeaway: Early tester involvement is a shift-left practice that prevents defects as well as detecting them.

Q15. Improving Incomplete Test Coverage

Correct results for common scenarios do not prove reliability; failures in rarely tested situations usually mean coverage is incomplete.

FINDING THE GAPS

Compare requirements with existing test cases. Gaps typically hide in functional requirements, negative scenarios, boundaries, error handling, integration paths, previously fixed defects and exceptional workflows.

CLOSING THEM WITH FORMAL TECHNIQUES

- **Equivalence Partitioning** – representative valid and invalid inputs
- **Boundary Value Analysis** – minimum and maximum values
- **Decision Tables** – combinations of conditions
- **State Transition** – behaviour across system states

Worked example: A field accepts 1 to 100. Test 0, 1, 2, 99, 100 and 101. Checking only 50 leaves the boundaries untested.

RECOMMENDED ACTIONS

- Build a requirement traceability matrix and list missing cases
- Add negative and boundary scenarios and past defects to regression
- Automate repetitive high-priority cases and review coverage before release

Takeaway: Full coverage cannot guarantee defect-free software, but systematic coverage analysis raises confidence in what has been validated.

Q16. Improving Defect Reporting

A report that lacks detail delays fixes because developers cannot reproduce the problem, which matters greatly in a logistics system spanning orders, inventory, warehouses and deliveries.

FIELDS OF A PROFESSIONAL REPORT

Field	Content
Defect ID / Title	Unique number and short summary
Environment	Browser, OS, application version
Preconditions	State required before execution
Steps to reproduce	Exact sequence of actions
Expected / Actual	Correct versus observed behaviour
Severity / Priority	Technical and business importance
Evidence	Screenshots, logs or video

Worked example: Replace “Delivery date is wrong” with the order number, delivery location, selected date, expected date, actual date, steps taken and application version.

New → Assigned → In progress → Fixed → Retest → Closed (reopened if the defect persists)

Benefits: reproducibility, communication, tracking, resolution speed, root-cause and historical analysis.

Takeaway: Standard, detailed reports turn a tester’s observation into usable technical information and speed up fixes.

Q17. Process-Based Testing Across Modules

When teams test modules by their own methods, results become inconsistent. A common process should apply everywhere, with extra testing added where risk is higher.

Requirement analysis → Test planning → Test case design → Environment setup → Execution → Defect reporting → Retest → Regression

STANDARDISE THE PAPERWORK

- One test-case format: ID, objective, preconditions, data, steps, expected result, actual result, status
- One defect-report format and common severity definitions

Worked example: In an e-commerce flow of Login → Search → Cart → Payment, every module follows the same process, but Payment gets extra security, transaction and integration testing because its risk is higher.

KEEPING TEAMS CONSISTENT

- Standard operating procedures and common templates
- Shared test-management tools and regression suites
- Test-case reviews and automation guidelines

Takeaway: *A shared process with risk-based additions improves traceability, repeatability and quality.*

Q18. Boundary Testing and Untested Edge Cases

Many systems work with normal input yet fail near limits or with unusual combinations. These edge cases are often missed because of incomplete requirements, limited test data, time pressure, a bias toward positive scenarios or no boundary analysis.

BOUNDARY VALUE ANALYSIS IN PRACTICE

Suppose an ATM allows a maximum daily withdrawal of ₹50,000.

Test value	Position
₹49,999	Just below the limit
₹50,000	Exactly at the limit
₹50,001	Just above the limit

These reveal more than an ordinary amount like ₹10,000.

OTHER EDGE CONDITIONS

- Empty inputs, zero, minimum and maximum values, maximum field length
- Invalid dates and expired information
- Duplicate requests and network interruption

Derive boundaries directly from the requirements, turn them into test cases, and keep them in regression after fixes.

Takeaway: *Systematic boundary and edge-case testing uncovers hidden defects and lowers the risk of production failures.*

Q19. Defect Documentation and Repository Management

Without proper documentation, an organisation forgets what went wrong, how it was reproduced and how it was fixed. A structured repository records each defect through its lifecycle.

Field	Purpose
Defect ID, Title	Identification and quick understanding
Module	Shows the affected area
Description, Steps	Explain and enable reproduction
Expected / Actual result	Define correct versus observed behaviour
Severity, Priority	Impact and urgency
Root cause, Resolution	Support prevention and record the fix
Status	Tracks the lifecycle

New → Assigned → In progress → Fixed → Retest → Closed (or reopened)

Analysis views: by module, severity, priority, release, root cause and recurring type. Repeated payment defects, for example, may signal weak transaction validation or integration.

Benefits: traceability, faster reproduction, communication, historical knowledge, root-cause analysis and stronger regression.

Takeaway: *The repository should be both a tracking system and a source of organisational learning.*

Q20. Redesigning Testing for Banking Transaction Processing

Repeated transaction failures show the current strategy is weak. Because money is involved, testing must cover correctness, integration, security, performance, boundaries and recovery.

Requirement analysis → Risk assessment → Test design → Functional → Integration → Performance → Security → Regression

CONDITIONS TO COVER

Valid transaction, insufficient balance, invalid account, invalid authentication, limit violation, duplicate transaction, network failure, server timeout, cancellation and recovery after failure.

Worked example: Account A holds ₹20,000 and sends ₹5,000 to Account B. Expect: ₹5,000 deducted from A, ₹5,000 credited to B, both balances correct, a single transaction record, the right status shown and a confirmation to the customer. If the network fails mid-process, there must be no duplicate or partial transaction.

TECHNIQUES APPLIED

- **Equivalence Partitioning** – valid and invalid inputs
- **Boundary Value Analysis** – transaction limits
- **Decision Tables** – account status, balance, amount, authorisation
- **State Transition** – initiated, processing, successful, failed, cancelled

IMPROVEMENTS TO INTRODUCE

- Automated regression, risk-based prioritisation and historical defect analysis
- Integration, performance, security, network-failure and recovery testing
- After each fix: retest the scenario, then regression-test related transaction functions

Takeaway: *A structured, risk-based approach with strong design, integration, boundary, performance and security testing makes transactions more reliable.*